

explain command design pattern(use java)

Command Design Pattern

The **Command Design Pattern** is a **behavioral design pattern** that encapsulates a request as an object.

In simple terms:

Convert an action/request into an object so that you can pass it around, store it, execute it later, undo it, or queue it.

It is useful when you want to **decouple the object that requests an operation from the object that actually performs it.**

1. Real-world example

Think about a **remote control**.

You press:

Remote Button → Turn ON TV



The remote doesn't need to know **how** the TV turns on.

It only knows:

Command → execute()



The actual TV logic is inside the command/receiver interaction.

```
graph TD; Invoker[Invoker] --> RemoteControl[RemoteControl]; RemoteControl --> Command[Command]; Command --> ConcreteCommand[ConcreteCommand]; ConcreteCommand --> TurnOnTVCommand[TurnOnTVCommand];
```

The diagram illustrates the Command Design Pattern structure. It shows a vertical flow of components: **Invoker** at the top, followed by **RemoteControl**, then **Command** (which includes the `execute()` method), then **ConcreteCommand**, and finally **TurnOnTVCommand** at the bottom. Arrows indicate the direction of interaction: from Invoker to RemoteControl, from RemoteControl to Command, from Command to ConcreteCommand, and from ConcreteCommand to TurnOnTVCommand. A vertical scrollbar is visible on the right side of the diagram area.

|
v
Receiver
TV



2. Main components

The Command pattern generally has **5 components**:

1. Command

Defines the operation.

```
interface Command {  
    void execute();  
}
```

</> Java



Sometimes it also defines:

</> Java



2. Concrete Command

Implements the command and knows which receiver should perform the operation.

```
class TurnOnCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOnCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override  
    public void execute() {  
        tv.turnOn();  
    }  
}
```

3. Receiver

The actual business object that performs the operation.

```
class TV {  
  
    public void turnOn() {  
        System.out.println("TV turned ON");  
    }  
  
    public void turnOff() {  
        System.out.println("TV turned OFF");  
    }  
}
```

The receiver contains the **real business logic**.

4. Invoker

The object that triggers the command.

```
class RemoteControl {  
  
    private Command command;  
  
    public void setCommand(Command command) {  
        this.command = command;  
    }  
  
    public void pressButton() {  
        command.execute();  
    }  
}
```

The `RemoteControl` doesn't know anything about `TV`.

It only knows:

</> Java



5. Client

Creates and wires the objects together.

```
public class Main {  
  
    public static void main(String[] args) {  
  
        TV tv = new TV();  
  
        Command turnOnCommand = new TurnOnCommand(tv);  
  
        RemoteControl remote = new RemoteControl();  
  
        remote.setCommand(turnOnCommand);  
  
        remote.pressButton();  
    }  
}
```

Output:

TV turned ON



3. Complete example

```
interface Command {  
    void execute();  
}
```

</> Java



```
class TV {  
  
    public void turnOn() {  
        System.out.println("TV turned ON");  
    }  
  
    public void turnOff() {  
        System.out.println("TV turned OFF");  
    }  
}
```

```
class TurnOnCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOnCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override  
    public void execute() {  
        tv.turnOn();  
    }  
}
```

```
class TurnOffCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOffCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override  
    public void execute() {  
        tv.turnOff();  
    }  
}
```

```
class RemoteControl {  
  
    private Command command;  
  
    public void setCommand(Command command) {  
        this.command = command;  
    }  
  
    public void pressButton() {  
        command.execute();  
    }  
}
```

Client:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        TV tv = new TV();  
  
        Command turnOn = new TurnOnCommand(tv);  
        Command turnOff = new TurnOffCommand(tv);  
  
        RemoteControl remote = new RemoteControl();  
  
        remote.setCommand(turnOn);  
        remote.pressButton();  
  
        remote.setCommand(turnOff);  
        remote.pressButton();  
    }  
}
```

Output:

```
TV turned ON  
TV turned OFF
```



4. Why do we need Command Pattern?

Consider this code without Command:

```
class RemoteControl {  
  
    private TV tv;  
  
    public void pressButton(String action) {  
  
        if (action.equals("ON")) {  
            tv.turnOn();  
        }  
        else if (action.equals("OFF")) {  
            tv.turnOff();  
        }  
    }  
}
```

Now imagine we add:

Light
Fan
AC
MusicSystem
Door



The remote starts becoming tightly coupled to everything:

```
if (action.equals("TV_ON")) {
    tv.turnOn();
}
else if (action.equals("LIGHT_ON")) {
    light.turnOn();
}
else if (action.equals("FAN_ON")) {
    fan.turnOn();
}
else if (action.equals("AC_ON")) {
    ac.turnOn();
}
```

This becomes difficult to maintain.

With Command:

```
RemoteControl
|
v
Command
|
+----- TurnOnTVCommand
|
+----- TurnOnLightCommand
|
+----- TurnOnFanCommand
|
+----- TurnOnACCommand
```



The remote doesn't care what operation is being performed.

5. Command Pattern decouples Invoker and Receiver

This is the **most important concept** for interviews.

Without Command:

```
Invoker -----> Receiver
Remote           TV
```



The remote directly knows about the TV.

With Command:

```
Invoker
|
v
Command
|
v
Receiver
```



For example:

```
RemoteControl
|
v
TurnOnCommand
|
v
TV
```



So:

Invoker doesn't directly depend on Receiver.

Instead, it depends on the `Command` abstraction.

6. Command as an object

This is the key idea behind the pattern.

Normally, we think of an operation as:

```
</> Java
```



Command converts that operation into an object:

</> Java



Now the operation itself can be:

- stored
- passed around
- queued
- logged
- retried
- scheduled
- executed later
- undone

For example:

```
List<Command> commands = new ArrayList<>();

commands.add(new TurnOnCommand(tv));
commands.add(new TurnOffCommand(tv));

for (Command command : commands) {
    command.execute();
}
```

This is one of the biggest advantages of the pattern.

7. Command Pattern with Undo

A common interview question is:

How can Command Pattern support undo?

Add:

</> Java



to the interface.

```
interface Command {  
  
    void execute();  
  
    void undo();  
}
```

</> Java

Example:

```
class TurnOnCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOnCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override  
    public void execute() {  
        tv.turnOn();  
    }  
  
    @Override  
    public void undo() {  
        tv.turnOff();  
    }  
}
```

Now:

```
command.execute();
```

</> Java

Output:

```
TV turned ON  
TV turned OFF
```

8. Command history

Because commands are objects, we can maintain command history.

```
class RemoteControl {

    private Command command;
    private final Stack<Command> history = new Stack<>();

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
        history.push(command);
    }

    public void undo() {
        if (!history.isEmpty()) {
            Command command = history.pop();
            command.undo();
        }
    }
}
```

Usage:

```
remote.setCommand(turnOn);
remote.pressButton();
```

This becomes:

```
execute()
  ↓
TV ON
  ↓
history.push(command)
  ↓
undo()
  ↓
TV OFF
```

9. Queueing commands

Another important use case is **command queues**.

For example:

```
Queue<Command> queue = new LinkedList<>();

queue.add(new TurnOnCommand(tv));
queue.add(new TurnOffCommand(tv));

while (!queue.isEmpty()) {
    queue.poll().execute();
}
```

This is useful in systems such as:

Message Processing
Job Queues
Task Scheduling
Background Processing
Batch Processing



The producer creates commands:

```
Producer
|
v
Command
|
v
Queue
|
v
Consumer
|
v
execute()
```



10. Lambda version in Java

Since Java supports functional interfaces, a simple Command can sometimes be represented using a lambda.

```
@FunctionalInterface  
interface Command {  
    void execute();  
}
```

</> Java



Then:

```
TV tv = new TV();  
  
Command turnOn = tv::turnOn;  
Command turnOff = tv::turnOff;  
  
turnOn.execute();  
turnOff.execute();
```

This is effectively treating the operation as a command object/function.

However, if you need additional state such as:

```
undo  
metadata  
logging  
authorization  
retry information  
command ID  
timestamps
```



a concrete command class is often more appropriate.

11. Practical backend example

Suppose you have an order system.

You have:

```
class OrderService {  
  
    public void createOrder() {  
        System.out.println("Order created");  
    }  
  
    public void cancelOrder() {  
        System.out.println("Order cancelled");  
    }  
}
```

```
}
```

Commands:

```
interface Command {  
    void execute();  
}
```

</> Java

```
class CreateOrderCommand implements Command {  
  
    private final OrderService orderService;  
  
    public CreateOrderCommand(OrderService orderService) {  
        this.orderService = orderService;  
    }  
  
    @Override  
    public void execute() {  
        orderService.createOrder();  
    }  
}
```

Now a job processor doesn't need to know how an order is created:

```
class JobProcessor {  
  
    public void process(Command command) {  
        command.execute();  
    }  
}
```

Usage:

```
OrderService orderService = new OrderService();  
  
Command command =  
    new CreateOrderCommand(orderService);  
  
JobProcessor processor = new JobProcessor();  
  
processor.process(command);
```

This architecture becomes useful when commands are being:

created
↓
queued
↓
processed
↓
executed
↓
logged
↓
retried



12. Command vs Strategy

This is a very common interview comparison.

Command

Encapsulates a **request/action**

Focuses on **what should be done**

Can support undo/history/queue

Commands can be stored and executed later

Example: CreateOrderCommand

Strategy

Encapsulates an **algorithm/behavior**

Focuses on **how something should be done**

Usually doesn't represent an individual request

Strategy is generally selected and used

Example: PaymentStrategy

Think:

Command → "Do this"
Strategy → "Do this using this approach"



13. Command vs Observer

Another useful distinction:

Command

Invoker → Command → Receiver




```
Command command = new CreateOrderCommand(orderService);

// execute later

</> Java
```



6. Supports retry

```
for (int i = 0; i < 3; i++) {
    try {
        command.execute();
        break;
    } catch (Exception e) {
        // retry
    }
}
```

15. Disadvantages

The main disadvantage is **additional classes**.

Instead of:

```
</> Java
```



you might have:

```
Command
TurnOnCommand
RemoteControl
TV
```



For a very simple application, this can be unnecessary complexity.

Also, if you have hundreds of operations, the number of command classes can increase significantly.

16. When should you use it?

Use Command when you need one or more of these:

- ✓ Decouple sender from receiver
- ✓ Queue operations
- ✓ Schedule operations
- ✓ Undo/redo
- ✓ Maintain command history
- ✓ Log operations
- ✓ Retry operations
- ✓ Execute operations asynchronously
- ✓ Treat operations as data/objects

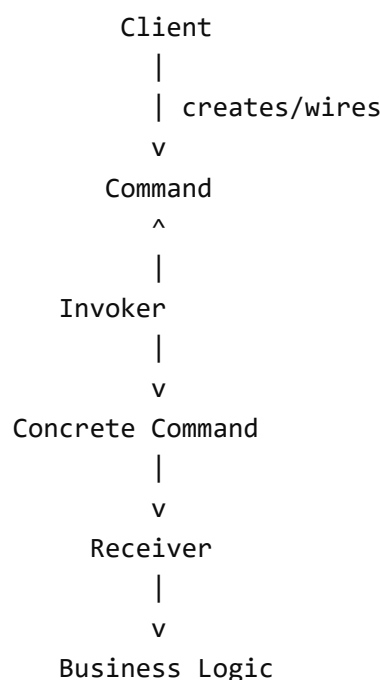


17. Interview definition

For a 7–8 year Java developer, I would explain it like this:

Command is a behavioral design pattern that encapsulates a request as an object, allowing the request to be parameterized, queued, logged, scheduled, retried, or undone, while decoupling the invoker from the receiver that performs the actual operation.

The core structure to remember is:



The most important interview points

1. Command encapsulates a request as an object.
2. Invoker triggers the command.
3. Receiver performs the actual business operation.
4. ConcreteCommand connects Command and Receiver.
5. Invoker is decoupled from Receiver.

6. Commands can be **queued, logged, scheduled, retried, and undone**.
7. In Java, a simple command can also be represented using a **functional interface + lambda/method reference**.
8. Don't confuse **Command** ("what action to perform") with **Strategy** ("which algorithm to use").



create above details as .md format test

Command Design Pattern

1. Definition

The **Command Design Pattern** is a **behavioral design pattern** that encapsulates a request as an object.

In simple terms:

Convert an action/request into an object so that it can be passed around, stored, executed later, queued, logged, retried, or undone.

The primary goal is to **decouple the object that requests an operation from the object that actually performs the operation**.

2. Real-World Example

Consider a **remote control**.

When you press a button:

Remote Button → Turn ON TV



The remote doesn't need to know how the TV turns on.

It only knows that it needs to execute a command:

```
RemoteControl
|
v
Command
```



```
execute()
  |
  v
TurnOnTVCommand
  |
  v
TV
```

The TV contains the actual business logic.

3. Intent

The primary intent of the Command pattern is:

Encapsulate a request as an object



This allows you to:

- Parameterize clients with different requests
- Queue requests
- Schedule requests
- Log requests
- Retry requests
- Support undo/redo
- Execute requests asynchronously
- Maintain command history

4. Structure

The Command pattern generally contains five major components:

```
Client
  |
  | creates
  v
Command
  ^
  |
Invoker
  |
  v
Concrete Command
  |
  v
Receiver
```



|
v
Business Logic

5. Components

5.1 Command

Defines the common operation that every command must implement.

```
public interface Command {  
  
    void execute();  
  
}
```

Sometimes it can also contain:

```
public interface Command {  
  
    void execute();  
  
    void undo();  
  
}
```

5.2 Concrete Command

A Concrete Command implements the `Command` interface.

It contains a reference to the **Receiver**.

```
public class TurnOnCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOnCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override
```

```
public void execute() {  
    tv.turnOn();  
}
```

The Concrete Command acts as a bridge between:

```
Invoker  
|  
v  
Command  
|  
v  
Receiver
```



5.3 Receiver

The Receiver is the object that actually performs the requested operation.

```
public class TV {  
  
    public void turnOn() {  
        System.out.println("TV turned ON");  
    }  
  
    public void turnOff() {  
        System.out.println("TV turned OFF");  
    }  
}
```

The Receiver contains the **actual business logic**.

5.4 Invoker

The Invoker is responsible for triggering the command.

```
public class RemoteControl {  
  
    private Command command;  
  
    public void setCommand(Command command) {  
        this.command = command;  
    }  
}
```

```
    }  
  
    public void pressButton() {  
        command.execute();  
    }  
}
```

Important:

The Invoker does **not** need to know about the Receiver.

It only knows about:

</> Java



and:

</> Java



5.5 Client

The Client creates and wires the objects together.

```
public class Main {  
  
    public static void main(String[] args) {  
  
        TV tv = new TV();  
  
        Command turnOnCommand = new TurnOnCommand(tv);  
  
        RemoteControl remote = new RemoteControl();  
  
        remote.setCommand(turnOnCommand);  
  
        remote.pressButton();  
    }  
}
```

Output:

TV turned ON



6. Complete Example

Command

```
public interface Command {  
  
    void execute();  
}
```



Receiver

```
public class TV {  
  
    public void turnOn() {  
        System.out.println("TV turned ON");  
    }  
  
    public void turnOff() {  
        System.out.println("TV turned OFF");  
    }  
}
```

Concrete Command - Turn ON

```
public class TurnOnCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOnCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override  
    public void execute() {  
        tv.turnOn();  
    }  
}
```


Concrete Command - Turn OFF

```
public class TurnOffCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOffCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override  
    public void execute() {  
        tv.turnOff();  
    }  
}
```

Invoker

```
public class RemoteControl {  
  
    private Command command;  
  
    public void setCommand(Command command) {  
        this.command = command;  
    }  
  
    public void pressButton() {  
        command.execute();  
    }  
}
```

Client

```
public class Main {  
  
    public static void main(String[] args) {  
  
        TV tv = new TV();  
  
        Command turnOn = new TurnOnCommand(tv);  
        Command turnOff = new TurnOffCommand(tv);  
  
        RemoteControl remote = new RemoteControl();  
  
    }  
}
```

```
        remote.setCommand(turnOn);  
        remote.pressButton();  
  
        remote.setCommand(turnOff);  
        remote.pressButton();  
    }  
}
```

Output:

```
TV turned ON  
TV turned OFF
```



7. Why Do We Need Command Pattern?

Without the Command pattern, the Invoker may become tightly coupled to multiple Receivers.

For example:

```
public class RemoteControl {  
  
    private TV tv;  
    private Light light;  
    private Fan fan;  
  
    public void pressButton(String action) {  
  
        if (action.equals("TV_ON")) {  
            tv.turnOn();  
        }  
        else if (action.equals("LIGHT_ON")) {  
            light.turnOn();  
        }  
        else if (action.equals("FAN_ON")) {  
            fan.turnOn();  
        }  
    }  
}
```

This has several problems:

- RemoteControl knows about every Receiver.
- Adding a new operation requires modifying RemoteControl .
- The class becomes difficult to maintain.

- The Invoker is tightly coupled to business objects.
- Violates the Open/Closed Principle to some extent.

With Command:

```

RemoteControl
  |
  v
Command
  |
  +----- TurnOnTVCommand
  |
  +----- TurnOnLightCommand
  |
  +----- TurnOnFanCommand
  
```



The remote doesn't need to know what operation is being performed.

8. Decoupling Invoker and Receiver

This is one of the most important concepts for interviews.

Without Command

```

Invoker -----> Receiver
RemoteControl      TV
  
```



The remote directly depends on the TV.

With Command

```

Invoker
  |
  v
Command
  |
  v
Receiver
  
```



For example:

```

RemoteControl
  |
  v
TurnOnCommand
  
```





|
v
TV

Therefore:

The Invoker does not directly depend on the Receiver.

Instead, the Invoker depends on the `Command` abstraction.

9. Command as an Object

The core idea of the pattern is to convert an operation into an object.

Normally:

</> Java



With Command:

</> Java



Now the operation itself can be:

- Stored
- Passed around
- Queued
- Logged
- Retried
- Scheduled
- Executed later
- Undone

For example:

```
List<Command> commands = new ArrayList<>();

commands.add(new TurnOnCommand(tv));
commands.add(new TurnOffCommand(tv));

for (Command command : commands) {
```

```
command.execute();  
}
```

This is one of the biggest advantages of the Command pattern.

10. Command Pattern with Undo

A common use case of Command is **undo/redo functionality**.

Modify the Command interface:

```
public interface Command {  
  
    void execute();  
  
    void undo();  
}
```

Concrete command:

```
public class TurnOnCommand implements Command {  
  
    private final TV tv;  
  
    public TurnOnCommand(TV tv) {  
        this.tv = tv;  
    }  
  
    @Override  
    public void execute() {  
        tv.turnOn();  
    }  
  
    @Override  
    public void undo() {  
        tv.turnOff();  
    }  
}
```

Now:

```
</> Java  
  
command.execute();
```

```
command.undo();
```

Produces:

TV turned ON
TV turned OFF



11. Command History

Because commands are objects, we can maintain command history.

</> Java



```
public class RemoteControl {  
  
    private Command command;  
  
    private final Stack<Command> history = new Stack<>();  
  
    public void setCommand(Command command) {  
        this.command = command;  
    }  
  
    public void pressButton() {  
  
        command.execute();  
  
        history.push(command);  
    }  
  
    public void undo() {  
  
        if (!history.isEmpty()) {  
  
            Command command = history.pop();  
  
            command.undo();  
        }  
    }  
}
```

Usage:

</> Java



```
remote.setCommand(turnOn);  
  
remote.pressButton();
```

```
remote.undo();
```

Execution flow:

```
pressButton()
  |
  v
command.execute()
  |
  v
TV ON
  |
  v
history.push(command)
  |
  v
undo()
  |
  v
command.undo()
  |
  v
TV OFF
```

12. Queueing Commands

Commands can also be placed into a queue.

```
</> Java

Queue<Command> queue = new LinkedList<>();

queue.add(new TurnOnCommand(tv));
queue.add(new TurnOffCommand(tv));

while (!queue.isEmpty()) {

    Command command = queue.poll();

    command.execute();
}
```

This is useful for:

- Job queues
- Message processing
- Background processing

- Task scheduling
- Batch processing
- Asynchronous processing

Architecture:

```

Producer
|
v
Command
|
v
Queue
|
v
Consumer
|
v
execute()

```



13. Scheduling Commands

Because a command is an object, it can be created now and executed later.

</> Java



```

Command command = new CreateOrderCommand(orderService);

// Execute Later
command.execute();

```

For example, a scheduler could maintain:

```

Command
|
+----- execute immediately
|
+----- execute after 5 minutes
|
+----- execute at midnight
|
+----- execute asynchronously

```



14. Java Lambda Implementation

Since Java supports functional interfaces, a simple Command can be represented using a lambda.

</> Java



```
@FunctionalInterface
public interface Command {

    void execute();
}
```

Now:

</> Java



```
TV tv = new TV();

Command turnOn = tv::turnOn;
Command turnOff = tv::turnOff;

turnOn.execute();
turnOff.execute();
```

This works because `Command` is a functional interface.

Equivalent lambda:

</> Java



```
Command turnOn = () -> tv.turnOn();
```

Method reference:

</> Java



```
Command turnOn = tv::turnOn;
```

15. When Should You Use a Concrete Command Class?

Lambda-based commands are useful for simple operations.

For example:

</> Java



```
Command command = () -> service.process();
```

However, a Concrete Command class is more appropriate when the command needs additional state or behavior.

For example:

```
Command
├─ commandId
├─ userId
├─ timestamp
├─ retryCount
├─ metadata
├─ execute()
├─ undo()
└─ validation
```

Example:

```
</> Java

public class CreateOrderCommand implements Command {

    private final OrderService orderService;
    private final Long orderId;

    public CreateOrderCommand(
        OrderService orderService,
        Long orderId) {

        this.orderService = orderService;
        this.orderId = orderId;
    }

    @Override
    public void execute() {
        orderService.createOrder(orderId);
    }
}
```

16. Practical Backend Example

Suppose we have an Order Management System.

Receiver

</> Java



```
public class OrderService {  
  
    public void createOrder() {  
        System.out.println("Order created");  
    }  
  
    public void cancelOrder() {  
        System.out.println("Order cancelled");  
    }  
}
```

Command

</> Java



```
public interface Command {  
  
    void execute();  
}
```

Concrete Command

</> Java



```
public class CreateOrderCommand implements Command {  
  
    private final OrderService orderService;  
  
    public CreateOrderCommand(OrderService orderService) {  
        this.orderService = orderService;  
    }  
  
    @Override  
    public void execute() {  
        orderService.createOrder();  
    }  
}
```

Invoker

</> Java



```
public class JobProcessor {  
  
    public void process(Command command) {
```

```
        command.execute();  
    }  
}
```

Client

```
</> Java  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        OrderService orderService = new OrderService();  
  
        Command command =  
            new CreateOrderCommand(orderService);  
  
        JobProcessor processor = new JobProcessor();  
  
        processor.process(command);  
  
    }  
}
```

Output:

= Order created [Upgrade](#)   ...

The important point is:

```
JobProcessor  
  |  
  v  
Command  
  |  
  v  
CreateOrderCommand  
  |  
  v  
OrderService
```

JobProcessor doesn't need to know how an order is created.

17. Command Pattern in Asynchronous Processing

Command is particularly useful in systems where operations need to be processed asynchronously.

Example:

```
REST API
  |
  v
Create Command
  |
  v
Message Queue
  |
  v
Consumer
  |
  v
Command.execute()
  |
  v
Business Service
```



For example:

```
POST /orders
  |
  v
CreateOrderCommand
  |
  v
Kafka / RabbitMQ / SQS
  |
  v
Order Consumer
  |
  v
OrderService
```



The command can represent the work that needs to be performed.

18. Command Pattern and Retry

Because the command encapsulates the operation, it can be retried.

</> Java

```
for (int i = 0; i < 3; i++) {

    try {

        command.execute();
```



```
        break;

    } catch (Exception e) {

        // Retry
    }
}
```

In production systems, retry logic would normally be handled by a dedicated mechanism such as:

- Spring Retry
- Resilience4j
- Message broker retry
- Dead-letter queues

The Command pattern itself does not provide retry; it simply makes the operation easier to represent and manage.

19. Command Pattern and Logging

Commands can be used to capture information about an operation.

For example:

```
Command ID
Command Type
User
Timestamp
Request Data
Status
Retry Count
```



Example:

```
Command:
    type      = CreateOrder
    commandId = 12345
    user      = user123
    timestamp = 10:30 AM
    status    = SUCCESS
```



This can be useful for:

- Auditing
- Debugging

- Monitoring
- Transaction history
- Operational logging

20. Command vs Strategy

This is a common interview question.

Command	Strategy
Encapsulates a request/action	Encapsulates an algorithm/behavior
Focuses on what should be done	Focuses on how something should be done
Can support undo/history/queue	Usually does not represent an individual request
Can be stored and executed later	Usually selected and used immediately
Example: <code>CreateOrderCommand</code>	Example: <code>PaymentStrategy</code>

Easy way to remember:

Command → "Do this"
Strategy → "Do this using this approach"



Example

Command:

```
</> Java
CreateOrderCommand
```



means:

Create this order.

Strategy:

```
</> Java
CreditCardPaymentStrategy
```



means:

Process payment using this payment algorithm.

21. Command vs Observer

Command

Represents a request to perform an operation.

```
Invoker
|
v
Command
|
v
Receiver
```



Example:

```
CancelOrderCommand
```



Observer

Represents notification when something happens.

```
Subject
|
+----> Observer 1
|
+----> Observer 2
|
+----> Observer 3
```



Example:

```
OrderCancelledEvent
|
+----> EmailService
|
+----> NotificationService
|
+----> AuditService
```



Easy distinction:

```
Command → Request something to happen
```


Observer → Notify that something happened



22. Advantages

22.1 Loose Coupling

The Invoker doesn't directly depend on the Receiver.

Invoker → Command → Receiver



22.2 Supports Undo/Redo

Commands can contain the information required to reverse an operation.

</> Java

```
command.execute();  
command.undo();
```



22.3 Supports Queueing

Commands can be placed into:

</> Java

```
Queue<Command>
```



and processed later.

22.4 Supports Logging

Commands can be recorded for auditing and debugging.

22.5 Supports Delayed Execution

A command can be created now and executed later.

22.6 Supports Retry

The same command can be executed again when an operation fails.

22.7 Supports Asynchronous Processing

Commands can be passed to asynchronous processing systems.

22.8 Follows Single Responsibility

The responsibilities are separated:

Invoker

→ triggers operation

Command

→ encapsulates request

Receiver

→ performs business operation



23. Disadvantages

23.1 More Classes

A simple operation:

`</> Java`

```
tv.turnOn();
```



could require:

Command

TurnOnCommand

RemoteControl

TV



This can introduce unnecessary complexity for very simple applications.

23.2 Increased Code

A large application may end up with many command classes:

```
CreateOrderCommand  
UpdateOrderCommand  
CancelOrderCommand  
DeleteOrderCommand  
ShipOrderCommand  
RefundOrderCommand  
...
```



23.3 Additional Abstraction

Developers need to understand:

```
Invoker  
Command  
ConcreteCommand  
Receiver  
Client
```



before understanding the complete flow.

24. When Should You Use Command Pattern?

Command is a good choice when you need:

- ✓ Decouple sender from receiver
- ✓ Queue operations
- ✓ Schedule operations
- ✓ Undo/redo
- ✓ Maintain command history
- ✓ Log operations
- ✓ Retry operations
- ✓ Execute operations asynchronously
- ✓ Treat operations as objects
- ✓ Pass operations between components



25. When Should You Avoid It?

Don't introduce Command merely because it is a design pattern.

For a simple operation:

```
</> Java
```



```
service.process();
```

creating:

```
Command  
ConcreteCommand  
Invoker  
Receiver
```



may be unnecessary.

Use it when the additional abstraction provides a real benefit such as:

- Queuing
- Undo
- Scheduling
- Decoupling
- Logging
- Retry
- Asynchronous execution

26. Important Interview Questions

Q1. What is the Command Design Pattern?

Answer:

Command is a behavioral design pattern that encapsulates a request as an object. It decouples the object that invokes an operation from the object that performs it and allows requests to be queued, logged, scheduled, retried, or undone.

Q2. What are the main components?

1. Command
2. ConcreteCommand
3. Receiver
4. Invoker
5. Client



Q3. Who contains the actual business logic?

Receiver.

Example:

```
</> Java   
  
class TV {  
  
    public void turnOn() {  
        // Actual business logic  
    }  
}
```

The Command delegates to the Receiver.

Q4. Does the Invoker know about the Receiver?

No.

The Invoker should generally know only about the `Command` abstraction.

```
</> Java   
  
class RemoteControl {  
  
    private Command command;  
  
    public void pressButton() {  
        command.execute();  
    }  
}
```

Q5. Why is the Command an object?

Because representing an operation as an object allows it to be:

```
stored  
queued  
passed around  
logged  
scheduled  
retried  
undone 
```

Q6. Can Command support undo?

Yes.

The interface can expose:

```
</> Java

void execute();

void undo();
```



Q7. Can Command be implemented using Lambda?

Yes, if the Command is a functional interface.

```
</> Java

@FunctionalInterface
interface Command {
    void execute();
}
```



Then:

```
</> Java

Command command = () -> service.process();
```



Q8. What is the difference between Command and Strategy?

```
Command → Encapsulates a request
Strategy → Encapsulates an algorithm
```



Q9. What is the main benefit of Command?

The primary benefit is:

Decoupling the Invoker from the Receiver while representing a request as an object.

27. Key Takeaways

Remember these points for interviews:

Command = Request represented as an object



Invoker → Command → Receiver



Command

Defines:

```
</> Java  
  
execute()
```



ConcreteCommand

Connects:

Command → Receiver



Receiver

Contains:

Actual business logic



Invoker

Calls:

```
</> Java  
  
command.execute();
```

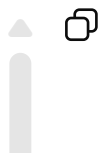


Client

Creates and wires everything together.

The biggest advantages are:

Decoupling
Queueing
Scheduling



Undo/Redo
Logging
Retry
Asynchronous execution
Command history



The easiest way to remember the pattern is:

"Instead of passing around the operation itself, make the operation an object."



ChatGPT can make mistakes. Check important info.